Business Thrust Techsoft Pvt. Ltd.
1st Floor, B-21, Block B, Industrial Area,
Sector 63, Noida, Uttar Pradesh-201301
www.bttechsoft.in
Tel: 0120-6618069

Order Management System

1. Assessment Overview

You are required to design and develop a **production-style Order Management System (OMS)** using:

Backend

- Laravel
- RESTful APIs
- MySQL
- Laravel Authentication
- Laravel Queues / Jobs where appropriate
- Laravel Events / Listeners where appropriate
- Proper validation and exception handling

Frontend

You may choose **ONE**:

- React.js
- Vue.js

You may use any supporting frontend libraries that you consider appropriate.

Important

AI tools are allowed and encouraged.

You may use:

- ChatGPT
- Claude
- GitHub Copilot
- Cursor
- Antigravity
- Gemini
- Other AI coding/research tools

However, candidates are expected to understand the code they submit and should be able to explain architectural and technical decisions during the review/interview.

The objective is to evaluate:

- Architecture
- Problem solving
- Full-stack development ability
- Database design
- API design

- Frontend architecture
- Code quality
- Security awareness
- Performance
- Testing
- Debugging
- Product/UX thinking
- Ability to use AI effectively

2. Business Scenario

Assume you are developing an Order Management System for a SaaS company.

The company receives orders from customers and manages the complete lifecycle from order creation to completion.

The system will be used by:

- Administrators
- Sales/Operations users
- Managers

Users should be able to:

1. Create customers
2. Create products
3. Create orders
4. Add products to orders
5. Assign orders to employees
6. Track order status
7. Add notes
8. View order activity
9. Filter and search orders
10. Monitor order performance from a dashboard
11. Manage users and permissions
12. View the complete history of changes

3. User Roles

Implement at least the following roles:

Admin

Can:

- Manage users
- Manage roles/permissions
- Manage customers
- Manage products

- Create/edit/delete orders
- Assign orders
- Change order status
- View all reports
- View audit logs

Manager

Can:

- View orders
- Create orders
- Edit orders
- Assign orders
- Change order status
- View dashboard
- View reports
- Add notes

Cannot:

- Manage system users
- Modify permissions

Staff

Can:

- View assigned orders
- Update allowed order statuses
- Add notes
- View customer/order information

Cannot:

- Manage users
- Delete orders
- Manage products

You may implement your own permission structure if you can justify it.

4. Authentication

Implement authentication using Laravel.

Minimum requirements:

- Login
- Logout
- Protected APIs

- Protected frontend routes
- Authenticated user information
- Role-based access

Optional:

- Forgot password
- Reset password
- Remember me
- Session/token management

The frontend must not rely only on hiding buttons for authorization.

Authorization must also be enforced at the backend/API level.

5. Order Management

Implement complete CRUD functionality for orders.

Create Order

The user should be able to:

- Select customer
- Add multiple products
- Specify quantities
- Set priority
- Set order date
- Assign employee
- Add notes
- Apply discount
- Calculate tax
- Calculate subtotal
- Calculate total

Example:

```
Product A
Quantity: 2
Unit Price: $100
Total: $200

Product B
Quantity: 3
Unit Price: $50
Total: $150

Subtotal: $350
Discount: $20
Tax: $33
Grand Total: $363
```

Calculations should be handled reliably.

Do not blindly trust totals sent from the frontend.

The backend should validate/recalculate important financial values.

6. Order Lifecycle

Implement the following order statuses:

```
Draft
Pending
Confirmed
Processing
Ready
Completed
Cancelled
```

You may add additional statuses if required.

However, implement sensible status transitions.

For example:

```
Draft
  ↓
Pending
  ↓
Confirmed
  ↓
Processing
  ↓
Ready
  ↓
Completed
```

Cancellation should be handled according to appropriate business rules.

The system should prevent invalid transitions.

For example:

```
Completed → Draft
Cancelled → Processing
```

should not be allowed unless you have explicitly designed a valid workflow for it.

7. Order Status History

Every status change must be recorded.

Example:

```
10:30 AM
John changed status

Pending → Confirmed

Comment:
Customer confirmed the order.
```

The order detail page should display the complete status history.

The history must include:

- Previous status
- New status
- User
- Date/time
- Optional remarks

8. Audit Logging

Implement an audit mechanism for important operations.

At minimum track:

- Order created
- Order updated
- Order deleted
- Status changed
- Assignment changed
- Customer updated
- Product updated

Example:

```
User: John

Action:
Updated Order #ORD-10234

Changed:
Priority:
Normal → High

Assigned To:
John → David

Timestamp:
```

Store old and new values where appropriate.

9. Customer Management

Create a customer management module.

Required functionality:

- Customer listing
- Search
- Filters
- Create customer
- Edit customer
- View customer
- Delete/deactivate customer

Customer detail page should show:

- Customer information
- Total orders
- Pending orders
- Completed orders
- Cancelled orders
- Total order value
- Recent orders

10. Product Management

Create a product management module.

Required:

- Product listing
- Search
- Filter
- Create product
- Edit product
- Product status
- Price
- Stock quantity
- SKU

Implement basic stock handling.

When an order is confirmed/processed, consider how stock should be affected.

You should define and document your approach.

For example:

```
Available Stock = 100

Order quantity = 5

Available Stock = 95
```

You must consider what should happen when an order is cancelled.

11. Dashboard

Create a professional dashboard.

At minimum show:

- Total Orders
- Pending Orders
- Confirmed Orders
- Processing Orders
- Completed Orders
- Cancelled Orders
- Total Revenue
- Average Order Value
- Total Customers
- Total Products

Add meaningful visualizations.

For example:

Order Status Distribution

```
Pending        120
Confirmed       85
Processing      63
Completed      320
Cancelled       25
```

Revenue Trend

Display revenue by:

- Day
- Week
- Month

The dashboard should use real API/database data.

Do not use hardcoded statistics.

12. Advanced Order Listing

Create a professional order listing page.

Required:

Search

Search by:

- Order number
- Customer name
- Customer email
- Product
- Assigned employee

Filters

At minimum:

- Status
- Priority
- Customer
- Assigned employee
- Date range

Sorting

Support sorting by:

- Order date
- Total
- Status
- Priority
- Customer

Pagination

Implement backend pagination.

Do not load thousands of records into the browser unnecessarily.

13. Order Detail Page

This is the most important screen.

The existing assessment also treats the order detail page as the critical screen.

It should contain:

Order Summary

- Order number
- Customer
- Order date
- Status
- Priority
- Assigned employee
- Total

Customer Information

- Name
- Email
- Phone
- Address

Order Items

Display:

- Product
- SKU
- Quantity
- Unit price
- Discount
- Tax
- Total

Timeline

Display:

```
Order Created
      ↓
Confirmed
      ↓
Processing
      ↓
Ready
      ↓
Completed
```

Activity

Show:

- Status changes
- Notes
- Assignment changes
- Important system activities

Actions

Depending on permissions:

- Edit
- Change Status
- Assign
- Add Note
- Cancel
- Delete

14. Create/Edit Order UI

Build a modern form.

The original task already expects logical field grouping, validation states, smart inputs and a wizard-style form.

You may implement:

Step 1

Customer information

Step 2

Products

Step 3

Pricing

Step 4

Assignment & Priority

Step 5

Review & Submit

Validation should exist on both frontend and backend.

Display useful validation messages.

15. Notifications

Implement an activity/notification system.

Notifications may include:

```
New order assigned to you.

Order #ORD-10021 has been confirmed.

Order #ORD-10031 was cancelled.

Order #ORD-10042 requires your attention.
```

Implement:

- Read
- Unread
- Mark as read
- Mark all as read

Optional:

- Real-time notifications
- WebSockets
- Laravel broadcasting

16. API Requirements

Build RESTful APIs.

Example:

```
POST   /api/login

GET    /api/orders
POST   /api/orders
GET    /api/orders/{id}
PUT    /api/orders/{id}
DELETE /api/orders/{id}

POST   /api/orders/{id}/status
POST   /api/orders/{id}/assign
POST   /api/orders/{id}/notes

GET    /api/customers
POST   /api/customers
GET    /api/customers/{id}
PUT    /api/customers/{id}

GET    /api/products
POST   /api/products
PUT    /api/products/{id}
```

You may structure the APIs differently if your design is better.

API responses should be consistent.

Example:

```
{
    "success": true,
    "message": "Order created successfully",
    "data": {}
}
```

Error responses should also be consistent.

17. API Validation

Implement proper Laravel validation.

Example:

```
Customer is required.

Order must contain at least one product.

Quantity must be greater than 0.

Product must exist.

Customer must exist.

Invalid status transition is not allowed.
```

Do not rely only on frontend validation.

18. Backend Architecture

Use a maintainable Laravel architecture.

We expect candidates to demonstrate understanding of:

- Controllers
- Form Requests
- API Resources
- Models
- Relationships
- Services where appropriate
- Policies / Gates
- Events / Listeners
- Jobs / Queues where appropriate
- Exception handling
- Database transactions

Avoid putting the entire business logic inside controllers.

For example, complex order creation should be designed carefully.

Consider:

```
Order Creation
     ↓
Validate Request
     ↓
Start Transaction
     ↓
Create Order
     ↓
Create Order Items
     ↓
Calculate Totals
     ↓
Update Inventory
     ↓
Create Activity
     ↓
Create Audit Log
     ↓
Commit Transaction
```

If any critical operation fails, the transaction should handle the failure appropriately.

19. Database Performance

Assume the application will eventually contain:

```
1,000,000+ orders
500,000+ customers
100,000+ products
5,000,000+ order items
```

The application should be designed with scalability in mind.

Candidates should consider:

- Database indexes
- Pagination
- Eager loading
- Avoiding N+1 queries
- Query optimization
- Proper relationships
- Select only required columns
- Filtering at database level

You do not need to actually create millions of records, but your implementation should demonstrate that you understand the implications.

20. Frontend Architecture

Use React OR Vue.

The frontend should have reusable components.

Examples:

```
Button
Modal
Input
Select
DatePicker
Badge
Table
Pagination
Dropdown
Toast
Loader
EmptyState
ErrorState
ConfirmationDialog
```

Do not duplicate the same UI implementation throughout the application.

The original assessment also emphasizes reusable buttons, cards, badges, inputs and modals as part of the design system.

21. Frontend State Management

Use an appropriate state management approach.

Examples:

React:

- Context
- Redux Toolkit
- Zustand
- React Query / TanStack Query

Vue:

- Pinia
- Vue Query

You should be able to explain why you selected your approach.

22. Loading / Empty / Error States

Every major screen must support:

Loading

Use skeleton loaders or an appropriate loading UI.

Empty

Example:

```
No orders found.

Try changing your filters or create a new order.
```

Error

Example:

```
Unable to load orders.

Try again.
```

The original assessment explicitly requires loading, empty and error states on major screens.

23. Security Requirements

Candidates should demonstrate awareness of:

- Authentication
- Authorization
- API protection
- Mass assignment protection
- SQL injection prevention
- XSS prevention
- CSRF where applicable
- Input validation
- File upload security if implemented
- Rate limiting where appropriate
- Sensitive data handling

Do not expose:

- Passwords
- Tokens
- Secrets
- Database credentials

24. Testing

Implement automated tests.

Minimum:

Backend

Create tests for:

- Authentication
- Order creation
- Order update
- Order status transition
- Invalid status transition
- Order validation
- Authorization
- Customer creation
- Product creation

Frontend

At least a reasonable number of tests for important components or workflows.

Candidates may use:

- PHPUnit / Pest
- PHPUnit Feature Tests
- Laravel HTTP tests
- Vitest
- Jest
- React Testing Library
- Vue Test Utils

The goal is to demonstrate understanding of what should be tested.

25. API Documentation

Provide API documentation.

You may use:

- Postman
- Swagger/OpenAPI
- Laravel API documentation tools

Documentation should include:

- Endpoint

- HTTP method
- Authentication
- Request parameters
- Request body
- Response
- Error response

Provide a Postman collection if possible.

26. Seed Data

Create realistic seed data.

Minimum recommended:

```
5 Admin/Manager/Staff users

100 Customers

50 Products

500+ Orders

Multiple order statuses

Multiple order priorities

Order notes

Order status history

Audit logs
```

The application should work immediately after database migration and seeding.

27. Code Quality

We will review:

- Naming conventions
- Folder structure
- Reusability
- Separation of concerns
- DRY principles
- SOLID principles
- Error handling
- API design
- Database design
- Component design
- Maintainability

Code that works but is unnecessarily difficult to maintain will not receive full marks.

28. Git Usage

Use Git throughout development.

Your repository should contain meaningful commits.

Avoid submitting everything in one commit such as:

```
final project
```

Good examples:

```
Initial Laravel setup

Create order database schema

Implement authentication APIs

Implement order CRUD

Implement order status workflow

Add dashboard APIs

Implement order listing UI

Add responsive design

Add automated tests
```

29. README

The repository must contain a detailed README.

Include:

Project Overview

Explain the application.

Technology Stack

Example:

```
Laravel
MySQL
React
Tailwind CSS
Redis
Docker
```

Installation

Explain setup.

Environment Configuration

Explain required environment variables without exposing secrets.

Database

Explain migrations and seeders.

API Documentation

Provide API documentation location.

Architecture

Explain important architectural decisions.

Trade-offs

Explain important decisions and alternatives considered.

AI Usage

Explain:

- Which AI tools were used
- Where AI was used
- What was generated with AI
- What was manually implemented/reviewed
- Any AI-generated code that required significant modification

30. AI Usage Policy

AI tools are **allowed**.

You are encouraged to use AI for:

- Research
- Debugging
- Boilerplate
- Code generation
- Refactoring
- Documentation
- Test generation
- UI ideas
- SQL optimization suggestions

However:

You are responsible for all submitted code.

During the technical discussion, you may be asked:

- Why did you choose this architecture?
- Why did you create this database structure?
- Why did you use this index?
- How does this API work?
- Why did you use a transaction here?
- What happens if two users update the same order?
- How would this scale to 1 million orders?
- Why did you choose React/Vue state management?
- What did the AI generate?
- Which AI-generated code did you change?
- What security risks exist?
- How would you improve this implementation?

Being unable to explain submitted code may negatively affect the evaluation.

31. Bonus Features

The following are optional and can increase the score:

Advanced UI

- Kanban order board
- Drag & drop
- Advanced filters
- Saved filters
- Bulk actions
- Grid/List toggle
- Custom dashboards

Backend

- Redis caching
- Queue processing
- Scheduled jobs
- Events/listeners
- Notifications
- WebSockets
- Real-time order updates

Business Features

- Order duplication
- Order export
- CSV export
- PDF invoice
- Customer order history
- Product stock alerts
- Low-stock notifications

Advanced Engineering

- Docker
- CI/CD
- Automated deployment
- API rate limiting
- Feature tests
- Static analysis
- Code formatting
- Performance optimizatio

32. Expected Deliverables

Candidate must submit:

1. GitHub Repository

Complete source code.

2. Database

- Migrations
- Seeders
- Factories

3. API

Complete Laravel API implementation.

4. Frontend

Complete React/Vue application.

5. API Documentation

Postman or Swagger/OpenAPI.

6. README

Complete setup and architecture documentation.

7. Technical Notes

A short document explaining:

- Architecture
- Major decisions
- Challenges
- Performance considerations
- Security considerations
- AI tools used

- Future improvements

33. Expected Timeline

Recommended assessment duration:

5 days

Candidates do not need to implement every optional feature.

The expectation is to deliver a **well-architected working MVP with strong core functionality** rather than an incomplete application containing every possible feature.

Quality is more important than the number of features.

34. What We Will Specifically Evaluate

We are not evaluating only whether the application works.

We will evaluate whether you can build a system that could realistically evolve into a production application.

We will look for:

Junior-level indicators

- Basic CRUD
- Basic Laravel APIs
- Basic React/Vue screens
- Basic database relationships

Mid-level indicators

- Good API architecture
- Proper validation
- Reusable frontend components
- Authentication
- Authorization
- Good database relationships
- Error handling
- Testing

Senior-level indicators

- Clear architecture
- Strong separation of concerns
- Transaction handling
- Proper status/state management

- Auditability
- Performance considerations
- Scalability
- Security
- Concurrency awareness
- Clean API contracts
- Meaningful automated tests
- Good technical decisions
- Ability to explain trade-offs

35. Interview / Code Review After Submission

After submission, the candidate may be invited for a technical discussion.

The discussion may include:

Laravel

- Explain your architecture.
- Why did you use Service classes?
- Where should business logic live?
- How do Laravel Policies work?
- How would you optimize a slow API?
- How would you handle queues?
- How would you handle transactions?

MySQL

- Explain your indexes.
- Why did you choose these indexes?
- What happens when the orders table reaches 10 million records?
- How would you investigate a slow query?
- Explain N+1 queries.
- How would you handle concurrent stock updates?

React/Vue

- Explain your state management.
- How are API calls managed?
- How do you prevent unnecessary API requests?
- How do you handle loading/error states?
- How is the component architecture organized?

System Design

- How would you scale this application?
- How would you introduce Redis?
- How would you implement real-time notifications?
- How would you handle 1,000 simultaneous order updates?
- How would you design the application for multi-tenancy?

36. Final Objective

Build a modern, production-oriented **Order Management System** that demonstrates your ability to work across:

```
Frontend
    ↓
React / Vue
    ↓
REST API
    ↓
Laravel
    ↓
Business Logic
    ↓
MySQL
    ↓
Caching / Queue / Events
```

The final system should demonstrate:

Good engineering + good UX + good architecture + good database design + good problem solving.

The use of AI is allowed.

The candidate, however, must demonstrate ownership and understanding of the final implementation.